

Best Django Setup for MediSync

Architecture brief for an AWS mentorship MVP

Recommended direction: a Django-first modular monolith with PostgreSQL, private S3 file storage, strict role-based access control, audit logs, and AWS App Runner deployment. Keep Next.js optional, not mandatory, for the first demo.

Project	MediSync - pharmacy intelligence and medication availability platform
Audience	Pharmacy staff, medication-search clients, possible doctors
Market	Lebanon-first pilot
Pilot scale	5 to 50 pharmacies
Team	Small student team
Delivery horizon	A few months
Data sensitivity	Prescriptions may be stored

Contents

1	Executive recommendation	3
1.1	Recommended stack	3
1.2	Why Django fits this project	3
1.3	Version choice	3
2	Architecture in one page	4
2.1	Target architecture	4
2.2	The main architectural rule	4
3	Frontend choice: Django templates vs React vs Next.js	4
3.1	Recommended MVP frontend	4
3.2	When to use React or Next.js	5
3.3	Practical recommendation	5
4	Django app structure	5
4.1	Project layout	5
4.2	Django apps and responsibilities	6
5	Database design	6
5.1	Database choice	6
5.2	Core entities	6
5.3	Important schema principles	7
6	Security model	7
6.1	Why security is a first-week concern	7
6.2	Roles	7
6.3	Required controls	8
6.4	Audit event examples	8
7	Files and prescriptions	8
7.1	Storage pattern	8
7.2	Prescription file rules	9
8	Imports and medicine-name matching	9
8.1	Why this matters	9
8.2	Recommended approach	9
8.3	Do not over-AI this	9
9	API design	9
9.1	Use DRF deliberately	9
9.2	Endpoint groups	10
9.3	Public availability endpoint	10
10	Deployment on AWS	10
10.1	Recommended AWS deployment	10
10.2	Why not ECS/Kubernetes now	11
11	Local development setup	11
11.1	Local services	11
11.2	Python dependencies	11
12	Settings strategy	12
12.1	Production settings checklist	12

13 Testing strategy	12
13.1 Minimum tests before demo	12
13.2 The one test you cannot skip	12
14 Build roadmap	13
14.1 Eight-week MVP plan	13
15 What to avoid	13
16 Final architecture decision	13

1 Executive recommendation

For this project, the best Django setup is not a microservices swarm, and it is not a heavy frontend/backend split unless the team can already execute that cleanly. The best setup is:

Best Django setup

Use **Django 5.2 LTS + Django REST Framework + PostgreSQL + private S3 storage + App Runner**. Build the MVP as a **modular monolith**. Use Django templates, HTMX, or a small React/Vite frontend for the dashboard. Add Next.js only if the team specifically needs SEO-heavy public pages or already knows it well.

1.1 Recommended stack

Layer	Recommended choice
Backend	Django 5.2 LTS, Django REST Framework, Django Admin
Frontend	Django templates + HTMX for fastest delivery; React/Vite or Next.js only if the team has frontend bandwidth
Database	PostgreSQL on Amazon RDS
File storage	Amazon S3 private bucket for prescription images, documents, imports, exports
Authentication	Django auth for MVP; optional Cognito integration later if AWS-native identity becomes a judging or production requirement
Authorization	Django groups/permissions plus object-level pharmacy scoping
Background jobs	Celery + Redis only for import processing, reports, expiry alerts, and future forecasting; avoid adding it before needed
Deployment	Dockerized Django app on AWS App Runner
Static files	WhiteNoise for app static files, or S3/CloudFront later
Monitoring	CloudWatch logs, structured application audit logs, optional Sentry
AI/Migration	RapidFuzz or similar fuzzy matching for medicine-name normalization; no generative medical advice in MVP

1.2 Why Django fits this project

Django is strong for MediSync because the MVP is mostly a secure business workflow: users, roles, pharmacies, medicines, stock, sales, invoices, prescription records, imports, and audit trails. Django includes an authentication system with users, groups, permissions, and sessions, which gives the team a stable base for role-based access control [3]. Django also has mature security documentation covering common web-application protections [4].

Django REST Framework fits when the system needs APIs for a public medication search, a future doctor portal, or a later mobile app. DRF includes permission classes and authentication support, so endpoints can be locked down per user role and per object scope [5].

1.3 Version choice

Use **Django 5.2 LTS**, not necessarily the newest Django release. Django 5.2 is designated as a long-term support release and receives security updates for at least three years after release [2]. For a student team with a health-adjacent project, long-term support is boring in exactly the right way. Boring is good here. Boring means fewer dragons in the walls.

2 Architecture in one page

2.1 Target architecture

Users

- |-- Pharmacy staff dashboard
- |-- Public/client medication availability search
- |-- Doctor portal, later
- |-- Admin/operations panel

Web layer

- |-- Django templates/HTMX, or React/Next.js if chosen
- |-- Django Admin for internal operations only

Django backend

- |-- accounts users, roles, pharmacy membership
- |-- pharmacies pharmacy profiles, branches, opening hours
- |-- catalog medicines, brands, generics, forms, strengths
- |-- inventory batches, stock movements, expiry, low stock
- |-- sales invoices, sales lines, returns
- |-- prescriptions prescription records and attachments
- |-- availability public availability status, no exact stock leak
- |-- imports CSV/Excel migration and medicine matching
- |-- audit who did what, when, from where
- |-- reports daily sales, expiry, low stock, margins

Data and infrastructure

- |-- PostgreSQL/RDS source of truth
- |-- S3 private prescription and import file storage
- |-- Redis optional job broker/cache
- |-- CloudWatch logs and metrics
- |-- App Runner managed deployment for the Django container

2.2 The main architectural rule

Do not split too early

Build a modular monolith first. Separate the code into Django apps, not separate deployed services. Microservices would add deployment, networking, tracing, authentication, and database consistency problems before the product has enough traffic to justify them.

At a 5 to 50 pharmacy pilot, a well-structured Django monolith with PostgreSQL can handle the workload. The real risks are not raw scale. The real risks are data leakage, messy permission rules, bad imports, poor UI workflow, and no auditability.

3 Frontend choice: Django templates vs React vs Next.js

3.1 Recommended MVP frontend

For a Django-first setup, the fastest path is:

Django templates + HTMX + small JavaScript where needed

This keeps the team focused on the pharmacy workflow rather than building two apps at once. You can still expose DRF APIs where useful, especially for autocomplete search, public availability search, and future mobile clients.

3.2 When to use React or Next.js

Option	Use it if	Avoid it if
Django templates + HTMX	You need the fastest working MVP, fewer moving parts, and strong back-end forms/admin flow	The team needs a highly interactive single-page app experience
React + Vite	You want a modern dashboard but do not need server-side rendering or SEO-heavy pages	The team is weak in frontend architecture
Next.js	You want a polished public portal, better SEO, landing pages, and separate app routes	The team is short on time and does not already know Next.js

3.3 Practical recommendation

Use this for the first mentorship build:

Django backend + Django templates/HTMX for staff dashboard
 DRF endpoints for search/autocomplete/API surfaces
 Django Admin for internal admin and data correction

Use this only if the team is comfortable with frontend work:

Next.js frontend + Django REST Framework backend

The Next.js version can look slicker, but it costs more coordination. The Django-template version is less shiny, but it gets the pharmacy workflow into users' hands faster.

4 Django app structure

4.1 Project layout

```
medisync/
  backend/
    manage.py
    pyproject.toml
    Dockerfile
    config/
      settings/
        base.py
        local.py
        production.py
      urls.py
      asgi.py
      wsgi.py
  apps/
    accounts/
    pharmacies/
    catalog/
    inventory/
    sales/
    prescriptions/
    availability/
    imports/
    audit/
    reports/
  tests/
```

```

frontend/                # optional if React or Next.js is used
infra/
  apprunner.yaml
  terraform/            # optional later
docs/
  architecture.md
  data-model.md
  security.md

```

4.2 Django apps and responsibilities

App	Responsibility
accounts	Custom user model, roles, pharmacy memberships, doctor accounts, login settings
pharmacies	Pharmacy profile, branch data, staff membership, opening hours, visibility status
catalog	Canonical medicine records, generic/brand names, dosage form, strength, manufacturer, search aliases
inventory	Batches, stock quantities, expiry dates, stock movements, supplier prices, reorder levels
sales	Invoices, invoice lines, returns, discounts, payment metadata, daily revenue reports
prescriptions	Prescription metadata, uploaded prescription files, prescription items, access control, verification status
availability	Public-facing availability status derived from inventory, with privacy rules
imports	CSV/Excel upload, column mapping, duplicate detection, fuzzy medicine matching, import review
audit	Immutable-ish event records for create/update/delete/view actions on sensitive resources
reports	Low stock, expiry risk, sales summaries, profit margin, future forecasting hooks

5 Database design

5.1 Database choice

Use **PostgreSQL**. MediSync is relational at its bones: pharmacies have staff, medicines have aliases and batches, invoices have lines, prescriptions have items, and availability is derived from stock. Amazon RDS is appropriate because it handles common database administration tasks such as setup, operation, scaling, backups, and patching [7]. RDS for PostgreSQL also supports backups, point-in-time restores, Multi-AZ deployments, read replicas, VPC placement, and SSL connections [8].

5.2 Core entities

Table	Purpose	Phase
users	Login identities; use a custom user model from day one	MVP
pharmacies	Pharmacy organization and public profile	MVP
pharmacy_memberships	User-to-pharmacy role mapping	MVP

<code>medicines</code>	Canonical medicine catalog	MVP
<code>medicine_aliases</code>	Brand names, misspellings, import aliases, generic links	MVP
<code>inventory_batches</code>	Quantity, batch number, expiry, supplier, cost	MVP
<code>stock_movements</code>	In/out adjustments and reason codes	MVP
<code>invoices</code>	Sale header	MVP
<code>invoice_lines</code>	Sale items	MVP
<code>prescriptions</code>	Prescription metadata and status	MVP
<code>prescription_items</code>	Medicines on the prescription	MVP
<code>availability_snapshots</code>	Public availability status derived from stock	MVP
<code>import_jobs</code>	Uploaded files, mapping, status, results	MVP
<code>audit_events</code>	Sensitive user actions	MVP
<code>doctor_profiles</code>	Optional doctor identity and verification	Phase 2

5.3 Important schema principles

- Use a **custom user model** from day one, even if it initially only adds basic fields.
- Use **pharmacy membership** instead of a single user role field, because a user may later work with multiple branches.
- Store **stock movements**, not only current quantity. Current quantity without movement history is a magic trick with no audit trail.
- Never expose exact stock quantity to public users by default. Show availability bands such as available, low stock, unavailable, or call pharmacy.
- Model prescriptions separately from sales. A prescription is a clinical/legal document; a sale is a commercial transaction.
- Avoid deleting sensitive records. Use soft-delete or status fields where legally appropriate.

6 Security model

6.1 Why security is a first-week concern

Because prescriptions may be stored, the system should be treated as sensitive from the first sprint. Django's security documentation emphasizes the usual web risks and the need to validate untrusted input [4]. For MediSync, that principle applies not only to forms, but also to CSV/Excel imports, medicine search, prescription uploads, filenames, and pharmacy profile fields.

6.2 Roles

Role	Can do	Cannot do
Platform Admin	Manage pharmacies, catalog, global settings, support workflows	Casually view prescriptions without audit reason
Pharmacy Owner	Manage own pharmacy, staff, stock, invoices, reports	Access other pharmacies
Pharmacist	Manage stock, sales, prescriptions for own pharmacy	Global admin actions

Pharmacy Staff	Limited sales and stock actions, depending on permissions	Sensitive reports unless granted
Doctor	Search availability; later create/view relevant prescriptions	View unrelated pharmacy internal stock
Public Client	Search medicine availability and pharmacy public details	See prescriptions, exact quantities, invoices

6.3 Required controls

- **Object-level permission checks:** every prescription, invoice, batch, and pharmacy record must be scoped to a pharmacy or authorized user.
- **Audit logs:** record who viewed, created, updated, deleted, exported, or downloaded sensitive data.
- **Private S3 storage:** prescription files should never be public. Use private buckets and short-lived signed URLs for access.
- **Encryption:** enable encryption for RDS and S3. Amazon S3 server-side encryption encrypts objects before saving them and decrypts them when downloaded [9].
- **No sensitive data in logs:** do not log prescription text, patient names, full IDs, or uploaded file contents.
- **Admin hygiene:** Django Admin should be restricted to trusted internal users and protected with strong authentication.
- **Upload validation:** restrict file types, scan file metadata, enforce file size limits, and store generated object keys instead of user filenames.
- **Backups:** enable database backups before any real pilot.
- **Legal review:** get local legal/privacy review before storing real prescription or patient-identifying data.

6.4 Audit event examples

```

PrescriptionViewed
PrescriptionCreated
PrescriptionAttachmentDownloaded
InventoryBatchAdjusted
InvoiceCreated
MedicineAliasMerged
ImportJobApproved
UserRoleChanged
AvailabilityPublished

```

Each audit event should include the actor, target object, pharmacy, timestamp, IP address or request metadata, and a safe summary. For sensitive events, include a reason field when appropriate.

7 Files and prescriptions

7.1 Storage pattern

Use S3 for prescription scans, uploaded Excel files, generated exports, and invoice PDFs. Use `django-storages` to connect Django's file storage API to S3; the package documents an Amazon S3 backend for Django file storage [10].

7.2 Prescription file rules

- Store files under non-guessable object keys, not under patient names.
- Store metadata in PostgreSQL: uploader, pharmacy, prescription status, file object key, checksum, MIME type, size, and timestamps.
- Access files through backend authorization checks and short-lived signed URLs.
- Separate prescription attachments from public images or static assets.
- Never put prescription files in the repository, local static folder, or public media path.

8 Imports and medicine-name matching

8.1 Why this matters

The MVP includes Excel/CSV migration and medicine-name cleaning. This is one of the highest-value demo features because pharmacies likely have messy naming conventions. It also creates the medicine availability data layer that the product vision needs.

8.2 Recommended approach

1. Upload CSV/Excel file.
2. Detect columns and show a mapping screen.
3. Parse rows into an import staging table.
4. Normalize text: lowercase, trim, remove duplicate whitespace, standardize dosage units.
5. Use exact match first.
6. Use alias match second.
7. Use fuzzy match third.
8. Require human approval for uncertain matches.
9. Create or update batches and stock movements only after approval.

8.3 Do not over-AI this

For the mentorship demo, the best “AI” feature is not a chatbot giving medical advice. It is a practical matching engine:

Input from import:	panadol tab 500
Suggested match:	Panadol 500 mg tablet
Confidence:	93%
Action:	Approve / choose another / create alias

This is useful, explainable, safer, and directly tied to pharmacy adoption.

9 API design

9.1 Use DRF deliberately

Django REST Framework should be used for endpoints that need to be consumed by JavaScript, a future mobile app, or external portals. DRF provides permission classes such as authenticated-only access and read-only unauthenticated access patterns [5].

9.2 Endpoint groups

```
/api/auth/
/api/me/
/api/pharmacies/
/api/catalog/medicines/
/api/inventory/batches/
/api/inventory/movements/
/api/sales/invoices/
/api/prescriptions/
/api/availability/search/
/api/imports/jobs/
/api/reports/
/api/admin/audit-events/
```

9.3 Public availability endpoint

The public search endpoint should not return internal stock numbers.

```
GET /api/availability/search/?q=panadol&area=beirut
```

Response:

```
{
  "medicine": "Panadol 500 mg tablet",
  "results": [
    {
      "pharmacy_name": "Example Pharmacy",
      "area": "Hamra",
      "status": "available",
      "last_updated": "2026-07-04T10:20:00Z"
    }
  ]
}
```

10 Deployment on AWS

10.1 Recommended AWS deployment

Use AWS App Runner for the Django backend. App Runner can deploy from source code or a container image and handles starting, running, scaling, and load balancing the service [6]. This is much simpler than ECS or Kubernetes for a student team.

AWS service	Use in MediSync
App Runner	Run the Django web container
RDS PostgreSQL	Store relational application data
S3	Store prescription attachments, import files, generated reports
CloudWatch	Logs and basic monitoring
Secrets Manager or SSM Parameter Store	Store database URL, secret key, API keys
ElastiCache Redis	Optional for Celery/cache after MVP needs it
CloudFront	Optional later for public frontend/static assets
Cognito	Optional later for AWS-native identity

10.2 Why not ECS/Kubernetes now

ECS and Kubernetes are valid for larger systems, but they add infrastructure work that does not help the first demo. The mentorship project should demonstrate architecture judgment, not cloud cosplay. App Runner plus RDS plus S3 is enough to show a serious AWS deployment.

11 Local development setup

11.1 Local services

Use Docker Compose locally for dependencies:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: medisync
      POSTGRES_USER: medisync
      POSTGRES_PASSWORD: medisync
    ports: ["5432:5432"]

  redis:
    image: redis:7
    ports: ["6379:6379"]

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    ports: ["9000:9000", "9001:9001"]
```

Redis and MinIO are optional at the very beginning. MinIO lets the team imitate S3 locally without uploading test prescription files to a real cloud bucket.

11.2 Python dependencies

A clean starter set:

```
Django==5.2.*
django-rest-framework
psycogp[binary]
gunicorn
whitenoise
django-environ
boto3
django-storages[s3]
openpyxl
pandas
rapidfuzz
drf-spectacular
pytest
pytest-django
factory-boy
ruff
```

Add only when needed:

```
celery[redis]
django-cors-headers
django-rest-framework-simplejwt
sentry-sdk
```

Simple JWT provides a JWT authentication backend for Django REST Framework [11]. Celery is a distributed task queue focused on real-time processing and scheduling [12]. Do not install either just to look impressive. Add them when the system actually needs them.

12 Settings strategy

Split settings by environment:

```
config/settings/base.py
config/settings/local.py
config/settings/production.py
```

12.1 Production settings checklist

- `DEBUG = False`
- Strong `SECRET_KEY` from Secrets Manager or SSM Parameter Store
- Strict `ALLOWED_HOSTS`
- HTTPS-only cookies
- CSRF trusted origins set correctly
- Secure HSTS settings after HTTPS is confirmed
- Database SSL enabled
- S3 private media storage
- Logs scrubbed of sensitive data

13 Testing strategy

13.1 Minimum tests before demo

Test type	What to test
Model tests	Stock movement calculations, expiry status, invoice totals
Permission tests	A pharmacy user cannot access another pharmacy's data
API tests	Search, import, prescription create/view/download workflows
Import tests	Duplicate detection, fuzzy matching thresholds, bad file handling
Security tests	Private file access, unauthenticated access rejection, role boundaries
Smoke tests	Login, create stock, create sale, upload prescription, search availability

13.2 The one test you cannot skip

Permission test

Create two pharmacies in the test database. Create users for each. Try to access the other pharmacy's inventory, prescriptions, invoices, and reports. Every request should fail. This test is the guard dog at the gate.

14 Build roadmap

14.1 Eight-week MVP plan

Week	Deliverable
1	Project setup, Django apps, PostgreSQL, custom user model, pharmacy model, local Docker setup
2	Authentication, role model, pharmacy membership, dashboard shell, admin restrictions
3	Medicine catalog, aliases, inventory batches, stock movements, low-stock and expiry basics
4	CSV/Excel import pipeline, staging screen, fuzzy medicine matching, manual approval
5	Sales and invoice recording, daily reports, basic margin view
6	Prescription record model, private attachment upload, audit logs, permission tests
7	Public availability search, availability snapshots, public pharmacy cards, no exact stock leakage
8	AWS deployment, seed demo data, monitoring, backup check, final polish and presentation

15 What to avoid

- **Do not** build full national e-prescription infrastructure for the MVP.
- **Do not** let AI provide diagnosis, dosage, substitution, or treatment advice.
- **Do not** use public S3 URLs for prescriptions.
- **Do not** expose exact pharmacy stock quantities to public users.
- **Do not** start with microservices.
- **Do not** build a mobile app unless the mentorship requirements force it.
- **Do not** skip object-level permission tests.
- **Do not** use Django Admin as the pharmacy staff UI forever; it is fine for internal operations and early data correction.

16 Final architecture decision

The best Django setup for MediSync is:

```
Django 5.2 LTS
Django REST Framework
PostgreSQL on Amazon RDS
Private Amazon S3 storage
Dockerized deployment on AWS App Runner
Django templates/HTMX for the MVP UI
Django Admin for internal operations
Optional Celery + Redis for imports/reports
Optional React/Next.js later for a richer public portal
```

References

- [1] User-provided project material. *MediSync Project Presentation*. Uploaded in the project conversation, July 2026.
- [2] Django Software Foundation. *Django 5.2 release notes*. <https://docs.djangoproject.com/en/6.0/releases/5.2/>
- [3] Django Software Foundation. *User authentication in Django*. <https://docs.djangoproject.com/en/6.0/topics/auth/>
- [4] Django Software Foundation. *Security in Django*. <https://docs.djangoproject.com/en/6.0/topics/security/>
- [5] Django REST Framework. *Permissions*. <https://www.django-rest-framework.org/api-guide/permissions/>
- [6] Amazon Web Services. *AWS App Runner documentation*. <https://docs.aws.amazon.com/apprunner/latest/dg/what-is-apprunner.html>
- [7] Amazon Web Services. *What is Amazon Relational Database Service?*. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>
- [8] Amazon Web Services. *Amazon RDS for PostgreSQL*. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html
- [9] Amazon Web Services. *Protecting data with encryption*. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html>
- [10] Django Storages. *Amazon S3 backend*. <https://django-storages.readthedocs.io/en/latest/backends/amazon-S3.html>
- [11] Simple JWT. *Simple JWT for Django REST Framework*. <https://django-rest-framework-simplejwt.readthedocs.io/>
- [12] Celery Project. *Celery - Distributed Task Queue*. <https://docs.celeryq.dev/>